

TELCO VISTA

Telecom Customer Analytics & Churn Prediction Platform

Technical & Functional Documentation

Built on Microsoft Azure | Azure Databricks | Power BI
Project Reference: NHA-4-087 | DEPI Data Engineering Track

Version 1.0

Table of Contents

Table of Contents.....	2
1. Executive Summary.....	4
2. Introduction & Business Problem	5
2.1 Business Context.....	5
2.2 The Problem.....	5
2.3 The Solution	5
3. Project Objectives	6
4. System Architecture Overview	7
4.1 Layered Architecture.....	7
4.2 High-Level Data Flow	7
4.3 Architecture Diagram (Textual).....	7
5. Technology Stack	8
6. Data Sources (Bronze Layer)	9
7. ETL Pipeline: Bronze to Silver.....	10
7.1 Profiling	10
7.2 Cleaning Rules by Table	10
7.3 Verification & Persistence.....	10
8. ETL Pipeline: Silver to Gold	11
8.1 Churn Label	11
8.2 Feature Engineering.....	11
8.3 Aggregations	11
8.4 Integration	11
9. Data Warehouse Design (Star Schema)	12
9.1 Entity-Relationship Diagram	12
9.2 Dimension Tables.....	12
9.3 Fact Table	12
9.4 Data Quality & Constraints	13
10. Machine Learning: Churn Prediction Model	14
10.1 Methodology.....	14
10.2 Feature Set.....	14
10.3 Notes on Results	14
11. Power BI Dashboard — Telco Vista Overview	15
11.1 Filters.....	15
11.2 KPI Cards	15
11.3 Visuals	15

11.4 Design Notes	16
11.5 Suggested Enhancements	16
12. Data Governance & Quality	17
12.1 Governance Model.....	17
12.2 Quality Controls Applied	17
13. Deployment & Reproduction Guide	18
14. Results & Key Insights	19
15. Limitations & Future Enhancements	20
16. Appendix A — gold_telecom Data Dictionary	21
17. Appendix B — Repository Structure	22

1. Executive Summary

Telco Vista is an end-to-end telecom customer analytics platform that turns raw operational data — customers, payments, calls, data consumption, network elements, mobile app activity, and loyalty configuration — into a governed data warehouse, a churn prediction model, and an executive Power BI dashboard. The platform was built on Microsoft Azure using Azure Databricks as the core data engineering and machine learning environment, following the medallion (Bronze / Silver / Gold) architecture.

The project was delivered as part of the DEPI (Digital Egypt Pioneers Initiative) Data Engineering track, under reference code NHA-4-087. It demonstrates a complete, production-style data pipeline: ingestion of eight raw source tables, systematic data cleaning and quality enforcement, feature engineering, a star-schema data warehouse for reporting, a PySpark-based churn classification model, and a Power BI dashboard that gives business stakeholders an at-a-glance view of the customer base, revenue, and churn risk.

9.93K Customers	52.39M Total Revenue	5.77 yrs Avg. Tenure	5% Churn Rate
---------------------------	--------------------------------	--------------------------------	-------------------------

This document describes the platform end-to-end: business context and objectives, system architecture, the data sources and their transformations, the data warehouse design, the machine learning methodology, the Power BI dashboard, data governance practices, and guidance for reproducing and extending the platform.

2. Introduction & Business Problem

2.1 Business Context

Telecom operators generate very large volumes of transactional and behavioral data — call detail records, data consumption logs, billing and payment events, mobile app interactions, and network telemetry. Individually, these datasets are operational by nature and difficult for business stakeholders to use directly. Telco Vista consolidates these sources into a single, trusted, analytics-ready view of the customer, enabling revenue reporting, customer segmentation, and churn management.

2.2 The Problem

Prior to this project, customer, billing, usage, and network data lived in separate operational tables with inconsistent quality (missing values, duplicates, inconsistent types, and outliers), with no unified customer-level table and no systematic way to identify which customers were at risk of churning. Business teams needed a single source of truth to answer questions such as: How many customers do we have, and where are they located? How much revenue are we generating, and from which segments? Which customers are likely to churn, and why?

2.3 The Solution

Telco Vista addresses this by building a governed medallion pipeline (Bronze → Silver → Gold) on Azure Databricks that cleans and integrates all eight source systems into one gold customer table; a star-schema warehouse optimized for reporting; a churn prediction model trained on the gold table; and a Power BI dashboard that surfaces the resulting KPIs, segments, and churn risk to decision-makers.

3. Project Objectives

- **Unify fragmented data.** Integrate eight raw operational sources (customer, payments, calls, consumption, consumption lookup, network elements, mobile app, loyalty configuration) into a single, clean, analytics-ready dataset.
- **Enforce data quality.** Systematically handle missing values, duplicates, inconsistent types, and outliers using a repeatable, auditable process.
- **Build a reporting-grade warehouse.** Model the integrated data as a star schema (fact and dimension tables) with primary/foreign key constraints suitable for BI tools.
- **Predict churn.** Train and evaluate a machine learning model that scores each customer's likelihood of churning, to support proactive retention.
- **Deliver actionable visibility.** Provide a self-service Power BI dashboard with the key KPIs, demographic and geographic breakdowns, and a churn-rate view.
- **Run on governed cloud infrastructure.** Use Microsoft Azure and Azure Databricks (Unity Catalog) so that data assets are centrally governed, secured, and reproducible.

4. System Architecture Overview

Telco Vista follows the medallion architecture, a layered approach that progressively refines data quality and structure as it moves through the pipeline. All compute and storage run on Azure Databricks, within the depiworkspace Unity Catalog, hosted on Microsoft Azure.

4.1 Layered Architecture

- **Bronze layer** — raw CSV extracts from the eight source systems, ingested as-is into a Databricks Volume with no transformation, preserving full fidelity to the source.
- **Silver layer** — cleaned, deduplicated, correctly typed Delta tables, one per source system, produced by the Bronze-to-Silver ETL notebook.
- **Gold layer** — a single wide, customer-grain table (gold_telecom) with engineered features (age, tenure, revenue aggregates, usage aggregates) and the CHURN label, produced by the Silver-to-Gold ETL notebook.
- **Warehouse layer** — a star schema (one fact table, four dimension tables) built from the gold table for efficient, governed BI reporting.
- **Consumption layer** — the PySpark churn model and the Power BI dashboard, both reading from the gold table / warehouse.

4.2 High-Level Data Flow

Raw source files → Bronze (Databricks Volume) → Silver (cleaned Delta tables) → Gold (gold_telecom) → branches into (a) the Telecom_DW star-schema warehouse and (b) the churn prediction model → both feed the Power BI dashboard for business consumption.

4.3 Architecture Diagram (Textual)

Bronze (8 raw tables) → ETL: bronze to silver.py → Silver (8 clean tables) → ETL: silver to gold.py → Gold (gold_telecom) → [Telecom_DW.sql → Star-Schema Warehouse] and [Model.py → Churn Model] → Power BI Dashboard (Telco Vista).

5. Technology Stack

Layer	Technology	Purpose
Cloud Platform	Microsoft Azure	Hosting for all compute, storage, and governance services
Data Engineering	Azure Databricks	Notebook-based ETL, Delta Lake storage, Unity Catalog governance
Processing Engines	PySpark, Pandas	Distributed and in-memory data cleaning & transformation
Storage Format	Delta Lake (Delta tables)	ACID-compliant, versioned storage for Bronze/Silver/Gold layers
Warehouse Modeling	SQL (Databricks SQL)	Star-schema DDL, constraints, and data quality fixes
Machine Learning	PySpark MLlib	Logistic Regression churn classifier with hyperparameter search
Visualization	Power BI	Executive dashboard: KPIs, breakdowns, map, churn gauge
Languages	Python, SQL	ETL, feature engineering, modeling, and warehouse scripting

6. Data Sources (Bronze Layer)

Eight raw operational tables are ingested into the Bronze layer. Representative samples of each are stored under `bronze_samples/` in the repository.

Source Table	Description	Key Fields
Customer	Customer master data: demographics, status, plan assignment	CUSTOMER_ID#, SUBSCRIBER_ID#, GENDER, BIRTHDATE, CUSTOMER_CLASS, SUBSCRIBER_TYPE
Payments	Billing / revenue transactions per subscriber	Payment_ID, SUBSCRIBER_ID#, RENT_REVENUE, IN_BUNDLE_REVENUE, TAX_AMOUNT
Calls	Call detail records	CALL_ID, SERVICE_NUMBER#, C_YEAR/C_MONTH/C_DAY, CALLTYPE, DURATION
Consumption	Daily data consumption records	Consumption_ID, SUBSCRIBER_ID#, DAILY_CONSUMPTION_MB, RATING_GROUP_ID#
Consumption RG Lookup	Rating group reference data	RATING_GROUP_ID#, Rating_Group_Name, Rating_Group_Type
Network Elements	Cell/cabin site attributes and geography	SERVICE_NUMBER#, CABIN_CODE, GOV, CABIN_LAT, CABIN_LONG, TECHNOLOGY_TYPE
Mobile App	Mobile self-care app interaction logs	LOG_ID, SERVICE_NUMBER#, ACTION_TYPE, DURATION_IN_SEC
Loyalty Configuration	Subscription-plan loyalty scheme reference data	SUBSCRIPTION_PLAN_ID#, MODEL, FAMILY, ACCUMULATION_FACTOR

The GOV (governorate) attribute reflects Egyptian administrative regions (e.g. Cairo, Alexandria, Dakahlia), which drive the geographic breakdowns shown later in the Power BI dashboard.

7. ETL Pipeline: Bronze to Silver

Implemented in ETL/bronze to silver.py, a Databricks notebook that loads each raw CSV with Pandas, profiles it, cleans it, and persists the result as a Delta table. The same repeatable pattern — profile, clean, verify, persist — is applied to all eight sources.

7.1 Profiling

Before any cleaning, every table is profiled: row counts, duplicate-row counts, and per-column null counts/percentages are printed, along with a full schema (dtypes) dump. This establishes a data-quality baseline for the Bronze layer.

7.2 Cleaning Rules by Table

- **Customer** — drop rows with a missing BIRTHDATE; fill missing GROUP_ID# with 'Unknown'; drop duplicates; trim/cast ID and text fields to string; uppercase SUBSCRIBER_STATUS and GENDER; parse STATUS_DATE, BIRTHDATE, and ACTIVATION_DATE to proper dates.
- **Payments** — fill missing revenue fields with 0.0; drop duplicates; cast IDs to string and revenue fields to float; parse CONNECT_DATE; remove RENT_REVENUE outliers using the IQR method (1.5× interquartile range).
- **Calls** — fill missing ADMIN_STATUS / OPERATION_STATUS / CALLTYPE with 'UNKNOWN' and missing DURATION with 0.0; drop duplicates; cast fields; construct a proper CALL_TIMESTAMP from C_YEAR / C_MONTH / C_DAY (mapping month names to numbers); remove DURATION outliers via IQR.
- **Consumption, Consumption Lookup & Loyalty** — fill missing RATING_GROUP_ID# with '0'; drop duplicates; normalize ID columns to trimmed strings across all three tables.
- **Network Elements** — fill missing ZIP_CODE with '00000' and missing CENTRAL_NAME with 'UNKNOWN'; drop duplicates; cast ID columns to string.
- **Mobile App** — fill missing PLAN_NAME / CATEGORY with 'UNKNOWN' and missing PRICE / SPEED / QUOTA with 0.0; drop duplicates; cast types accordingly.

7.3 Verification & Persistence

After cleaning, each table is re-checked for remaining nulls (the notebook confirms zero nulls across all eight tables) and previewed (schema + sample rows). Each cleaned Pandas DataFrame is then converted to Spark and written as a managed Delta table under depiwkworkspace.default: customer_clean, payments_clean, calls_clean, network_clean, mobile_clean, consumption_clean, consumption_lookup_clean, and loyalty_clean.

8. ETL Pipeline: Silver to Gold

Implemented in ETL/silver to gold.py. This stage engineers features, computes aggregates, and integrates all Silver tables into a single customer-grain Gold table.

8.1 Churn Label

CHURN is derived as a binary flag: CHURN = 1 when SUBSCRIBER_STATUS equals 'SUSPENDED', otherwise CHURN = 0.

8.2 Feature Engineering

- AGE — computed as the number of years between today and BIRTHDATE.
- TENURE_DAYS — computed as the number of days between today and ACTIVATION_DATE.
- Raw PII and date columns no longer needed downstream (NAME, ID_TYPE, GROUP_ID#, SUBSCRIBER_STATUS, STATUS_DATE, BIRTHDATE, ACTIVATION_DATE) are dropped after derivation.

8.3 Aggregations

Source	Grouped by	Aggregated measures
Payments	SUBSCRIBER_ID#	TOTAL_RENT, TOTAL_IN_BUNDLE, TOTAL_OUT_BUNDLE, TOTAL_DEVICES, TOTAL_ADDON, TOTAL_TAX, AVG_IN_BUNDLE, NUM_PAYMENTS
Calls	ACCOUNT_ID#	TOTAL_CALLS, AVG_CALL_HOUR
Consumption	SUBSCRIBER_ID#	TOTAL_CONSUMPTION_MB, AVG_DAILY_CONSUMPTION_MB, TOTAL_CONSUMPTION_RECORDS
Mobile App	CUSTOMER_ID#	TOTAL_MOBILE_ACTIONS, TOTAL_MOBILE_DURATION, AVG_MOBILE_DURATION

8.4 Integration

The customer base is left-joined with the payments, consumption, mobile app, and calls aggregates (on SUBSCRIBER_ID#, CUSTOMER_ID#, and ACCOUNT_ID# respectively), plus network attributes (CABIN_CODE, CENTRAL_NAME, GOV, TECHNOLOGY_TYPE, coordinates) joined on SERVICE_NUMBER#. The result is written as depiworkspace.default.gold_telecom — the single analytics-ready table that powers the warehouse, the churn model, and the Power BI dashboard.

9. Data Warehouse Design (Star Schema)

Implemented in Data Warehouse/Telecom_DW.sql, built from gold_telecom (and loyalty_clean for the plan dimension), then promoted from the default schema into a dedicated depiworkspace.telecom schema.

9.1 Entity-Relationship Diagram

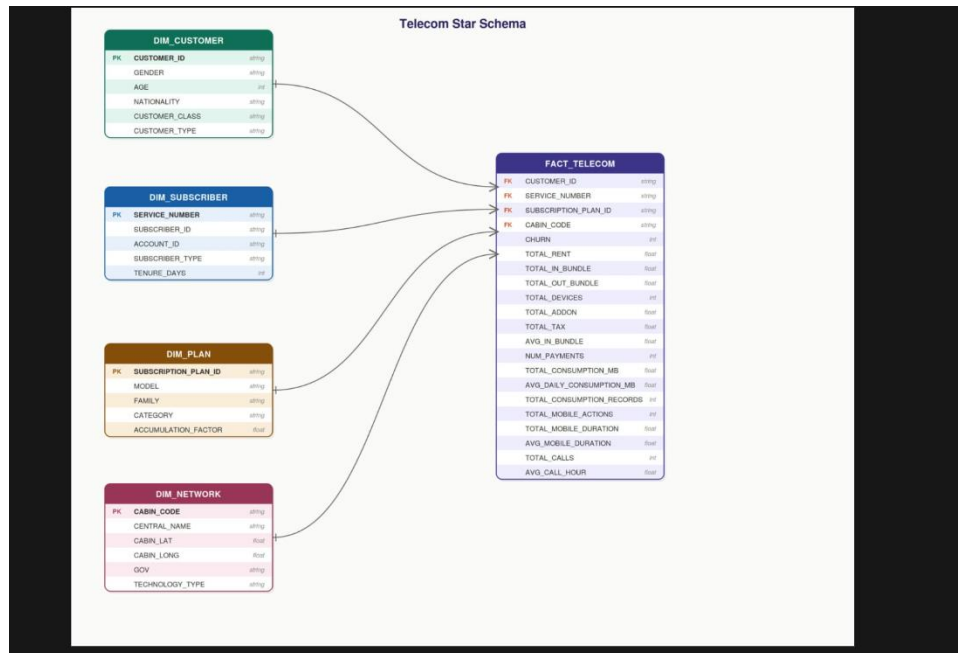


Figure 1 — Telco Vista star schema: fact_telecom linked to dim_customer, dim_subscriber, dim_network, and dim_plan.

9.2 Dimension Tables

Table	Grain	Key Columns
dim_customer	1 row per customer	CUSTOMER_ID (PK), GENDER, AGE, NATIONALITY, CUSTOMER_CLASS, CUSTOMER_TYPE
dim_subscriber	1 row per subscriber	SERVICE_NUMBER (PK), SUBSCRIBER_ID, ACCOUNT_ID, SUBSCRIBER_TYPE, TENURE_DAYS
dim_network	1 row per network cabin	CABIN_CODE (PK), CENTRAL_NAME, CABIN_LAT, CABIN_LONG, GOV, TECHNOLOGY_TYPE
dim_plan	1 row per subscription plan	SUBSCRIPTION_PLAN_ID (PK), MODEL, FAMILY, CATEGORY, ACCUMULATION_FACTOR

9.3 Fact Table

fact_telecom holds one row per customer, with foreign keys to all four dimensions (CUSTOMER_ID, SERVICE_NUMBER, SUBSCRIPTION_PLAN_ID, CABIN_CODE) plus the CHURN flag and every revenue and usage measure produced in the Gold layer (TOTAL_RENT, TOTAL_IN_BUNDLE,

TOTAL_OUT_BUNDLE, TOTAL_DEVICES, TOTAL_ADDON, TOTAL_TAX, AVG_IN_BUNDLE, NUM_PAYMENTS, TOTAL_CONSUMPTION_MB, AVG_DAILY_CONSUMPTION_MB, TOTAL_CONSUMPTION_RECORDS, TOTAL_MOBILE_ACTIONS, TOTAL_MOBILE_DURATION, AVG_MOBILE_DURATION, TOTAL_CALLS, AVG_CALL_HOUR).

9.4 Data Quality & Constraints

- 77 rows with a NULL CABIN_CODE were re-pointed to a synthetic 'UNKNOWN' network record instead of being dropped, preserving fact-table completeness while keeping referential integrity.
- NOT NULL and PRIMARY KEY constraints were added to every dimension's natural key.
- FOREIGN KEY constraints link fact_telecom to all four dimensions.
- Row-count and duplicate-key checks (GROUP BY ... HAVING COUNT(*) > 1) were run on each dimension before promoting the schema.
- Access to the warehouse schema was explicitly granted to the project's Databricks service accounts.

10. Machine Learning: Churn Prediction Model

Implemented in Model.py using PySpark MLlib, trained directly on gold_telecom.

10.1 Methodology

- **1. Feature selection** — identifier columns not useful as predictors (SERVICE_NUMBER#, SUBSCRIBER_ID#, ACCOUNT_ID#, CUSTOMER_ID#, CABIN_CODE) are dropped.
- **2. Train/test split** — 80% / 20%, with a fixed random seed (42) for reproducibility.
- **3. Categorical encoding** — StringIndexer (handleInvalid='keep') applied to GENDER, NATIONALITY, CUSTOMER_CLASS, SUBSCRIBER_TYPE, TECHNOLOGY_TYPE, and GOV.
- **4. Feature assembly** — VectorAssembler combines the indexed categoricals with 17 numeric features covering demographics, tenure, revenue, and usage.
- **5. Class imbalance handling** — a per-row class weight (ratio of negative to positive churn cases) is computed and used as the model's weight column.
- **6. Model & tuning** — Logistic Regression with a grid search over regParam {0.0, 0.01} × elasticNetParam {0.0, 1.0}, selecting the best combination by ROC-AUC on the held-out test set.
- **7. Evaluation** — Accuracy, weighted Precision, weighted Recall, F1 score, ROC-AUC, and a confusion matrix (actual vs. predicted CHURN).

10.2 Feature Set

Category	Features
Categorical	GENDER, NATIONALITY, CUSTOMER_CLASS, SUBSCRIBER_TYPE, TECHNOLOGY_TYPE, GOV
Numeric	AGE, TENURE_DAYS, TOTAL_IN_BUNDLE, TOTAL_OUT_BUNDLE, TOTAL_DEVICES, TOTAL_ADDON, TOTAL_TAX, AVG_IN_BUNDLE, NUM_PAYMENTS, TOTAL_CONSUMPTION_MB, AVG_DAILY_CONSUMPTION_MB, TOTAL_CONSUMPTION_RECORDS, TOTAL_MOBILE_ACTIONS, TOTAL_MOBILE_DURATION, AVG_MOBILE_DURATION, TOTAL_CALLS, AVG_CALL_HOUR

10.3 Notes on Results

Exact metric values (AUC, accuracy, precision, recall, F1) depend on the full production dataset processed in Azure Databricks and are captured in the notebook's run output; the small CSV samples shipped in the repository are for structural reference only and are not representative of full-scale performance.

11. Power BI Dashboard — Telco Vista Overview

The dashboard is built on the warehouse (depiworkspace.telecom) or directly on gold_telecom, and gives a single-page executive view of the customer base.

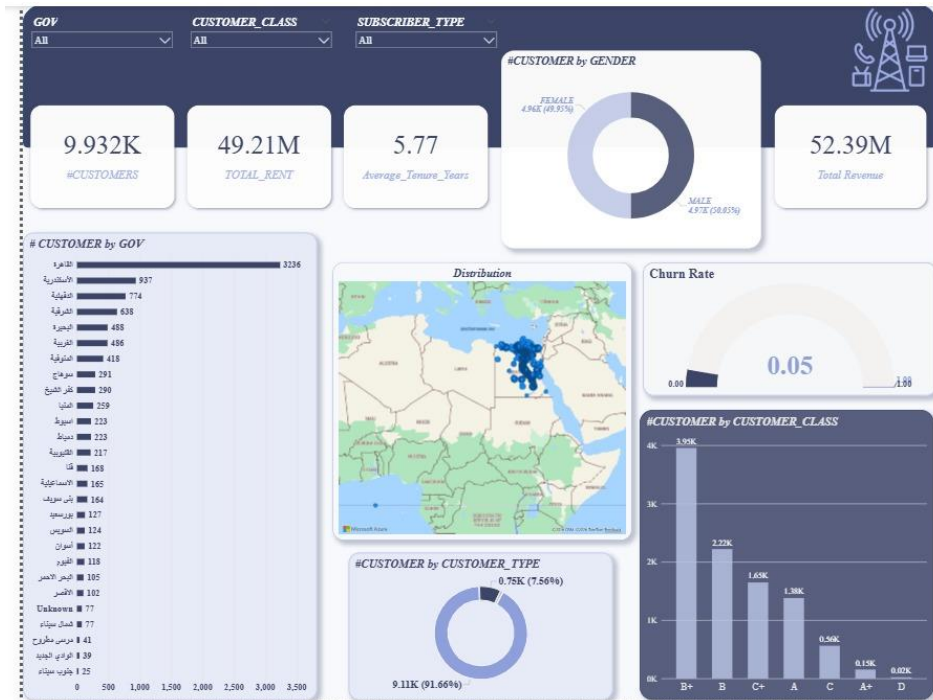


Figure 2 — Telco Vista Power BI dashboard: filters, KPI cards, demographic and geographic breakdowns, and churn-rate gauge.

11.1 Filters

Slicer	Source Column	Purpose
GOV	dim_network.GOV	Filter by governorate / region
CUSTOMER_CLASS	dim_customer.CUSTOMER_CLASS	Filter by customer class
SUBSCRIBER_TYPE	dim_subscriber.SUBSCRIBER_TYPE	Filter by subscriber type

11.2 KPI Cards

KPI	Definition	Value shown
#CUSTOMERS	Distinct customer count	9.932K
TOTAL_RENT	Sum of rent revenue	49.21M
Average_Tenure_Years	Average tenure in years (TENURE_DAYS / 365)	5.77
Total Revenue	Sum of all revenue streams	52.39M

11.3 Visuals

- **#CUSTOMER by GENDER** — donut chart; near-even split (Female 49.95%, Male 50.05%).
- **#CUSTOMER by GOV** — ranked horizontal bar chart of customer count per governorate; Cairo dominates (3,236), followed by Alexandria (937) and Dakahlia (774).
- **Distribution** — geographic map plotting network/customer locations from CABIN_LAT / CABIN_LONG, clustered around Cairo and the Nile Delta.
- **Churn Rate** — gauge showing overall share of customers with CHURN = 1; currently 0.05 (5%).
- **#CUSTOMER by CUSTOMER_TYPE** — donut chart; the base is overwhelmingly one customer type (91.66% vs. 7.56%).
- **#CUSTOMER by CUSTOMER_CLASS** — column chart across class tiers (B+, B, C+, A, C, A+, D), with B+ and B dominating.

11.4 Design Notes

All visuals are cross-filtered by the three top slicers, so any governorate/class/subscriber-type selection updates every card, chart, and the map together. The map and the GOV bar chart intentionally present the same geography dimension from two angles: spatial density and ranked magnitude.

11.5 Suggested Enhancements

- Add a trend view (churn rate / revenue over time) once a date dimension is introduced to the fact table.
- Add a drill-through page from the churn gauge to a customer-level list of at-risk customers, using scores from the churn model.
- Add revenue-per-customer and ARPU-by-GOV measures to connect revenue KPIs with the geographic breakdown.
- Localize labels for a fully bilingual (Arabic/English) audience.

12. Data Governance & Quality

12.1 Governance Model

All data assets are managed within Azure Databricks' Unity Catalog under the `depiworkspace` catalog, giving centralized access control, lineage tracking, and auditability across the Bronze, Silver, Gold, and warehouse layers.

12.2 Quality Controls Applied

- Null handling — explicit fill or drop rules per column, tailored to each source table (see Section 7.2).
- Deduplication — `drop_duplicates()` applied to every Silver table.
- Type enforcement — explicit casting to string, float, or datetime for all identifier, measure, and date columns.
- Outlier control — IQR-based filtering ($1.5\times$ interquartile range) applied to `RENT_REVENUE` and `call DURATION`.
- Referential integrity — primary and foreign key constraints enforced across the warehouse's fact and dimension tables.
- Verification gates — the Bronze-to-Silver notebook re-checks null counts after cleaning and confirms zero remaining nulls before writing Silver tables.

13. Deployment & Reproduction Guide

To reproduce the platform end-to-end:

- 1. Provision an Azure Databricks workspace (Unity Catalog enabled) on Microsoft Azure.
- 2. Load the eight raw tables into a Databricks Volume at `/Volumes/depiworkspace/default/telecom_data`.
- 3. Run ETL/bronze to silver.py to produce the cleaned `*_clean` Delta tables.
- 4. Run ETL/silver to gold.py to build the unified `gold_telecom` table.
- 5. Run Data Warehouse/Telecom_DW.sql to build the star-schema warehouse (`depiworkspace.telecom`).
- 6. Run Model.py to train and evaluate the churn prediction model on `gold_telecom`.
- 7. Connect Power BI to the Azure Databricks warehouse (or directly to `gold_telecom`) and refresh the Telco Vista dashboard.

14. Results & Key Insights

- **Customer base** — roughly 9,930 active customers, evenly split by gender, generating a combined 52.39M in total revenue.
- **Geographic concentration** — Cairo alone accounts for the largest single share of customers, followed by Alexandria and Dakahlia; the map view confirms most network sites cluster around Cairo and the Nile Delta.
- **Customer mix** — the base is dominated by one customer type (over 91%) and by the B+ and B customer classes, suggesting a concentrated core segment worth prioritizing for retention offers.
- **Churn** — the overall churn rate stands at roughly 5%, low in absolute terms but still representing a meaningful base of at-risk revenue given total customer volume; the churn model provides a customer-level score to focus retention efforts.
- **Data quality** — the ETL layer eliminated all detected null values and clear outliers across all eight source tables before promotion to the Gold layer, giving the warehouse and the model a consistently clean foundation.

15. Limitations & Future Enhancements

- The churn label is derived purely from `SUBSCRIBER_STATUS = 'SUSPENDED'`; it does not distinguish voluntary vs. involuntary churn, nor does it account for reactivations.
- Only Logistic Regression is evaluated; tree-based models (Random Forest, Gradient-Boosted Trees) could be benchmarked for potential accuracy gains.
- The repository ships only sample slices of each source table for portability; the production pipeline runs against the full Volume-hosted CSVs.
- No automated CI/tests currently cover the ETL or model scripts.
- The dashboard is currently a single point-in-time snapshot; adding a date dimension would enable trend analysis over time.
- Future iterations could add a drill-through 'at-risk customers' page, ARPU-by-region measures, and full Arabic/English localization.

16. Appendix A — gold_telecom Data Dictionary

Category	Columns
Identifiers	CUSTOMER_ID#, SUBSCRIBER_ID#, ACCOUNT_ID#, SERVICE_NUMBER#, CABIN_CODE
Demographics	GENDER, AGE, NATIONALITY, CUSTOMER_CLASS, SUBSCRIBER_TYPE
Tenure	TENURE_DAYS
Target	CHURN (0 / 1)
Revenue	TOTAL_RENT, TOTAL_IN_BUNDLE, TOTAL_OUT_BUNDLE, TOTAL_DEVICES, TOTAL_ADDON, TOTAL_TAX, AVG_IN_BUNDLE, NUM_PAYMENTS
Usage	TOTAL_CONSUMPTION_MB, AVG_DAILY_CONSUMPTION_MB, TOTAL_CONSUMPTION_RECORDS, TOTAL_MOBILE_ACTIONS, TOTAL_MOBILE_DURATION, AVG_MOBILE_DURATION, TOTAL_CALLS, AVG_CALL_HOUR
Network	CENTRAL_NAME, CABIN_LAT, CABIN_LONG, GOV, TECHNOLOGY_TYPE

17. Appendix B — Repository Structure

Path	Contents
bronze_samples/	Sample raw CSVs for the 8 source tables
ETL/bronze to silver.py	Cleaning: nulls, duplicates, types, outliers (IQR)
ETL/silver to gold.py	Feature engineering, aggregation, CHURN label, gold table
Data Warehouse/Telecom_DW.sql	Star-schema DDL, constraints, data quality fixes
Data Warehouse/*.png	Entity-relationship diagram
Model.py	PySpark churn prediction (Logistic Regression + grid search)
README.md	Project overview and quick-start guide
docs/DOCUMENTATION.md	Full technical documentation (Markdown)
docs/POWER_BI.md	Power BI dashboard documentation (Markdown)

End of Document — Telco Vista Documentation, v1.0